VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JNANA SANGAMA" , BELAGAVI-590 018, KARNATAKA

Project Report
on

"Multi Channel Notification Delivery System"

BACHELOR OF ENGINEERING

Submitted By

Vivek G	**AF05066531**
Ankitha N	**AF05060865**
Harshitha	**AF05066537**

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING

SAI VIDYA INSTITUTE OF TECHNOLOGY

(Affiliated to Visvesvaraya Technological University, Belagavi | Recognized by Govt. of Karnataka | Approved by AICTE, New Delhi)

Accredited by NBA, New Delhi (CSE, ISE, ECE), NAAC 'A' Grade

RAJANUKUNTE, BENGALURU – 560 064

2025-26

PARADOX STUDENT SELF-CONSISTENCY &INTEGRITY ANALYZER

INDEX

Chapter No. Title

1 **Introduction**

2 **Objectives of the Project**

3 **Scope of the Project**

4 **Feasibility Study**

5 **System Design**

6 **Existing application**

7 **Proposed application**

8. **ER Diagram**

9. **Screenshots of database**

10. **Coding and your application screenshots**

11. **Testing and Validation**

12. **Conclusion**

ABSTRACT

The Multi-Channel Notification Delivery System is a robust and scalable platform designed to deliver notifications to users through multiple communication channels such as email, SMS, and push notifications. The system ensures that critical information reaches users reliably and efficiently, regardless of their preferred mode of communication.

In modern applications, timely and consistent communication is essential, but relying on a single notification channel can lead to delivery failures or delays. This project addresses that challenge by implementing a unified system capable of sending notifications across multiple channels simultaneously or selectively, based on user preferences and system configurations.

The system is developed using technologies such as Spring Boot, Hibernate, and MySQL, ensuring a strong backend architecture and efficient data handling. It incorporates intelligent routing mechanisms to determine the most effective delivery channel, along with failover strategies that automatically switch to alternative channels if the primary one fails. This enhances reliability and guarantees message delivery.

Additionally, the system supports features such as message scheduling, delivery tracking, and user preference management, allowing for personalized and efficient communication. Security measures are also implemented to ensure safe transmission of data.

The primary objective of this project is to design a flexible, efficient, and reliable notification system that can be integrated into various applications such as banking systems, e-commerce platforms, and educational portals. By improving communication reliability and user engagement, the system contributes to better user experience and operational efficiency.

CHAPTER 1

INTRODUCTION

1.1 Introduction to the Project

In the modern digital world, communication between systems and users plays a vital role in ensuring seamless interaction and user engagement. Almost every application today—whether it is an e-commerce platform, banking system, healthcare application, or social media service—relies heavily on notifications to convey important information. These notifications can be in the form of emails, SMS messages, in-app alerts, or system-to-system webhooks.

Despite their importance, implementing a reliable notification system is a challenging task. Developers often need to integrate multiple third-party services separately for each communication channel, which increases system complexity and development effort. Moreover, these systems frequently lack proper failure handling mechanisms. Notifications may fail due to temporary issues such as network failures or service downtime, and without a proper retry mechanism, such failures can lead to loss of critical information.

To overcome these challenges, the Multi-Channel Notification Delivery Platform is designed as a centralized and scalable backend solution that enables applications to send notifications through multiple channels using a single unified interface. The platform is developed using Java 17 and Spring Boot, leveraging modern backend practices to ensure high performance, reliability, and maintainability.

Moreover, traditional notification systems frequently lack proper failure handling mechanisms. Notifications may fail due to temporary issues such as network failures, server downtime, API rate limits, or service unavailability. Without a proper retry mechanism and fallback strategy, such failures can lead to loss of critical information, which may negatively impact user experience and business operations.

Another major challenge is the absence of a centralized system to manage and monitor notifications. In many applications, notification logic is scattered across different modules, making it difficult to track delivery status, debug issues, or maintain consistency. This results in poor system maintainability and reduced efficiency.

1.2 Background of the Project

Traditional communication methods such as emails and SMS have long been used for delivering notifications to users. However, with the increasing demand for faster and more reliable communication, modern systems are shifting towards advanced notification mechanisms that support multiple channels. Applications today require instant delivery of alerts, updates, and transactional messages to ensure better user engagement and system efficiency.

Modern notification systems offer features such as:

- Multi-channel delivery (Email, SMS, Push Notifications)
- Real-time notification processing
- Automated retry mechanisms
- Delivery status tracking
- Scalable and distributed communication

However, many existing notification systems are either too complex, tightly coupled with specific services, or require high infrastructure and maintenance costs. Additionally, most third-party notification services are dependent on external APIs, making integration and customization difficult for small-scale or academic projects.

This creates an opportunity to develop a custom notification system that:

- Supports multiple communication channels in a unified way
- Provides reliable notification delivery
- Is easy to understand and implement
- Can be extended for future enhancements

With the rapid advancement of internet technologies, communication has evolved from simple message delivery systems to highly efficient and intelligent notification platforms. Modern applications rely heavily on notifications for critical operations such as OTP verification, transaction alerts, reminders, and system updates. However, these systems often depend on complex architectures and external services, making them less suitable for educational purposes or beginner-level implementation.

In an educational context, it is important to understand how notification systems work internally rather than just using third-party services. This project provides an opportunity to explore the design and development of a **Multi-Channel Notification Delivery System** using modern technologies like Spring Boot and Hibernate.

By developing this system from scratch, the project enables learners to gain practical knowledge of how real-world notification systems operate, how failures are handled using retry mechanisms, and how multiple communication channels can be managed efficiently within a single platform.

1.3 Problem Statement

Despite the availability of numerous communication platforms and notification services, there is a need for a simple, customizable, and efficient notification system that demonstrates real-world software development concepts while ensuring reliable message delivery across multiple channels.

The key problems identified are:

- Lack of a unified system to manage multiple notification channels (Email, SMS, Push)
- Complexity in integrating different third-party notification services
- Failure of notifications due to network issues or service downtime
- Absence of proper retry mechanisms and fallback strategies
- Difficulty in monitoring and tracking notification delivery status
- Limited understanding of multi-channel communication systems among students

Therefore, this project aims to develop a **Multi-Channel Notification Delivery System** that addresses these issues by providing a centralized, scalable, and structured platform for sending notifications efficiently across multiple communication channels.

1.4 Purpose of the Project

The primary purpose of this project is to design and develop a **Multi-Channel Notification Delivery System** that enables efficient and reliable communication between applications and users through multiple communication channels such as Email, SMS, and Push Notifications.

In modern applications, timely delivery of notifications is critical for ensuring user engagement and system effectiveness. However, relying on a single communication channel often leads to delivery failures and reduced reliability. This project aims to overcome these limitations by providing a unified platform that can send notifications through multiple channels, thereby increasing the chances of successful message delivery.

Furthermore, the system focuses on simplifying the complexity involved in managing multiple notification services. Instead of integrating different APIs separately, the platform provides a centralized interface that handles notification routing, delivery, and tracking. This makes the system easier to maintain, extend, and use in different applications.
The system aims to:

- Provide a centralized platform for sending notifications
- Enable communication through multiple channels
- Improve reliability using retry mechanisms
- Simplify integration with external services
- Enhance user engagement through timely alerts

1.5 Features of the System

The **Multi-Channel Notification Delivery System** includes several features that enhance reliability, efficiency, and usability of notification delivery across multiple platforms.

1. **Multi-Channel Notification Support**

   • Sends notifications through multiple channels such as Email, SMS
   • Ensures higher delivery success rate

2. **Notification Routing System**

   • Automatically selects appropriate channel(s) for delivery
   • Can send notifications through single or multiple channels

3. **Retry Mechanism**

   • Automatically retries failed notifications
   • Reduces chances of message loss

4. **Notification Logging and Tracking**

   • Maintains records of all notifications sent
   • Tracks delivery status (Success / Failed / Pending)

- **User Management**

  • Stores user details such as email and phone number
  • Manages user preferences for notification channels

- **Centralized Notification System**

  • Provides a single interface for sending notifications
  • Eliminates need to integrate multiple APIs separately

- **Database Management**

  • Stores notification data securely using MySQL
  • Uses Hibernate ORM for efficient data handling

1.6 Technologies Used

The project is developed using the following technologies:

Backend:

- Spring Boot
- Java

Frontend:

- HTML
- CSS
- JavaScript

Database:

- MySQL

Tools:

- Spring Tool Suite (STS)
- Maven

1.7 Advantages of the System

- High Reliability
- Improved Communication
- Centralized Management
- Scalable System
- Easy Integration
- Reduced Failure Rate

1.8 Applications of the System

The **Multi-Channel Notification Delivery System** can be used in various domains:

- Banking Systems
- E-commerce Platforms
- Educational Institutions
- Healthcare Systems

1.9 Importance of Secure Communication Systems

In modern computing environments, data security plays a crucial role in notification systems. With the increasing number of cyber threats, unauthorized access, and data breaches, it has become essential to protect user information and communication. Notification systems often handle sensitive data such as OTPs, transaction details, and personal alerts, making them a potential target for attackers.

The implementation of security mechanisms ensures that the data transmitted through different channels remains safe and protected. Although notifications may not always require heavy encryption like messaging systems, secure handling of data during transmission and storage is necessary to maintain confidentiality and prevent misuse.

This project incorporates secure communication practices at the application level to ensure that user data is protected. Proper validation, controlled access, and secure API integration are used to prevent unauthorized operations. These measures help in safeguarding the system from common vulnerabilities and attacks.

1.10 Role of Web-Based Applications

Web-based applications play a significant role in modern software systems by providing easy accessibility and platform independence. Unlike traditional desktop applications, web applications can be accessed from any device with an internet connection, without the need for installation or configuration. This makes them highly convenient and user-friendly.

In the context of the **Multi-Channel Notification Delivery System**, a web-based approach ensures that the system can be accessed and managed efficiently from different locations. Administrators can monitor notifications, track delivery status, and manage system operations through a simple web interface.

1.11 Need for Custom Chat Applications

In modern applications, relying on a single communication channel is no longer sufficient to ensure reliable and effective message delivery. Users expect instant notifications across different platforms, and any delay or failure in communication can negatively impact user experience and system performance. This creates a strong need for multi-channel notification systems.

Multi-channel notification systems allow messages to be delivered through multiple communication channels such as Email, SMS, and Push Notifications. This approach increases the chances of successful delivery, as the system can use alternative channels if one fails. For example, if an email is not delivered due to server issues, the same notification can be sent via SMS or push notification.

1.12 Challenges in Developing Chat Applications

Developing an efficient notification system involves several challenges that must be addressed to ensure reliable and smooth communication. As modern applications require real-time and multi-channel communication, handling these complexities becomes an important aspect of system design.

One of the major challenges is managing multiple communication channels. Each channel, such as Email, SMS, and Push Notifications, has its own APIs, configurations, and limitations. Integrating and maintaining these different services in a single system increases development complexity.

Another significant challenge is ensuring reliable message delivery. Notifications may fail due to network issues, server downtime, API rate limits, or service provider failures. Without proper handling mechanisms such as retries or fallback options, important messages may be lost.

1.13 Conclusion of Introduction

The **Multi-Channel Notification Delivery System** is an effective solution designed to improve communication between systems and users by utilizing multiple notification channels. It addresses the limitations of traditional single-channel systems by providing a reliable, scalable, and centralized platform for sending notifications.

CHAPTER 2

OBJECTIVES OF THE PROJECT

2.1 Primary Objective

The primary objective of this project is to design and develop a **Multi-Channel Notification Delivery System** that enables efficient and reliable communication by sending notifications through multiple channels such as Email, SMS, and Push Notifications using a centralized platform.

2.2 Secondary Objectives

1. To Develop a User-Friendly System

The system aims to provide a simple and easy-to-use interface for managing and sending notifications.

2. To Implement Multi-Channel Communication

The project enables sending notifications through multiple channels to improve delivery success rate.

3.To Ensure Reliable Notification Delivery

The system includes mechanisms to handle failures and ensure that notifications are delivered successfully.

4.To Integrate Backend Technologies

The project demonstrates the integration of Spring Boot and Hibernate for efficient backend processing.

5.To Maintain Notification Records

The system stores notification details and delivery status for tracking and monitoring purposes**.**

6.To Provide Practical Learning Experience

The project helps in understanding real-world concepts such as API integration, database management, and system design**.**

7. To Build Scalable Architecture

The system is designed in a modular way so it can be extended with new features in the future.

2.3Advanced Objectives

1.To Implement Retry Mechanism

The system automatically retries failed notifications to ensure reliability**.**

2. To Design Scalable Architecture

The system is designed in a way that it can handle increasing number of users.

3.To Enable Efficient Data Handling

The system uses Hibernate ORM for effective database interaction and data management.

4.To Support Multiple Notification Channels

Allows easy addition of new channels like WhatsApp or in-app notifications.

5.To Support Future Enhancements

Designed in a modular way to easily add new features in the future.

6.To Enable Centralized Management

Provides a single platform to manage all notification services.

2.4 Functional Objectives

The functional objectives define the main operations performed by the system. These include sending notifications, managing users, processing requests, and tracking delivery status. The system ensures that notifications are processed and delivered efficiently across different channels.

2.5 Non-Functional Objectives

Non-functional objectives focus on system performance and quality. These include:

- High reliability
- Fast response time
- Data security
- Scalability
- User-friendly design

These objectives ensure that the system not only works correctly but also provides a good user experience.

CHAPTER 3

SCOPE OF THE PROJECT

3.1 Introduction to Project Scope

The scope of the **Multi-Channel Notification Delivery System** defines the functional boundaries, technical capabilities, and limitations of the system. It describes what the system is designed to perform and the range of its operations.

This project focuses on developing a centralized platform that enables applications to send notifications through multiple communication channels such as Email, SMS, and Push Notifications. The system is designed as a learning-oriented application that demonstrates real-world notification handling while maintaining simplicity and efficiency.

The scope also includes system design, data handling, user interaction, and integration with external services. However, it does not extend to highly complex real-time distributed systems or large-scale enterprise-level infrastructure.

3.2 Functional Scope

3.2.1 User and Notification Management

The system manages user details and notification requests. It stores user information such as email ID and phone number, which are required for sending notifications.

Functional capabilities include:

- Storing user details
- Managing notification requests
- Associating notifications with users

3.2.2 Multi-Channel Notification Delivery

The system supports delivering notifications through multiple channels, ensuring higher reliability.

Supported channels:

- Push notifications
- SMS alerts
- Email notifications

3.2.3 Notification Processing System

The system processes incoming notification requests and determines how the message should be delivered.

Feature include:

- Message formatting
- Channel selection
- Request handling
- Sending notifications

3.2.4 Retry and Failure Handling

The system includes a retry mechanism to handle failed notifications.

Features:

- Automatic retry for failed messages
- Limited retry attempts
- Failure logging

This improves the reliability of the system.

3.2.5 Notification Tracking and Logging

The system maintains records of all notifications sent.

Capabilities include:

- Tracking delivery status
- Logging successful and failed messages
- Monitoring system performance

3.2.6 User Interface Scope

The application includes:

- Attractive chat interface
- Responsive layout
- Easy navigation
- Message display formatting

3.3 Technical Scope

3.3.1 System Architecture

The system follows a **3-tier architecture**:

1. **Presentation Layer**
   Handles user interface and input

2. **Business Logic Layer**
   Processes notification requests using Spring Boot

3. **Data Layer**
   Stores data using MySQL database

This architecture improves scalability and maintainability

3.3.2 Backend Capabilities

The backend performs core system operations such as:
• Processing notification requests
• Integrating with external APIs
• Managing retry mechanisms
• Handling business logic

3.3.3 Database Scope

The database stores important system data such as:

• User information
• Notification details
• Delivery status logs
• Channel information

The use of Hibernate ORM ensures efficient database operations.

3.3.4 Integration Scope

The system can integrate with:

• Email service providers
• SMS gateways
• Push notification services
• External APIs

This makes the system flexible and extendable.

3.4 User-Based Scope

3.4.1 Users

Users are the primary recipients of notifications in the system. They interact with the system indirectly by receiving alerts, updates, and important messages through different communication channels such as Email, SMS, and Push Notifications.

Users can:
• Receive notifications through multiple channels
• Get real-time updates and alerts
• Stay informed about important events and activities
• Experience improved communication reliability

3.4.2 Administrator

The administrator plays a crucial role in managing and controlling the system. The admin is responsible for handling system operations, monitoring performance, and ensuring smooth functioning of the notification delivery process.

Administrator capabilities include:

• Managing user data and details
• Sending and controlling notifications
• Monitoring delivery status of notifications
• Viewing logs of successful and failed messages
• Managing system configurations

3.5 Operational Scope

The **Multi-Channel Notification Delivery System** can be applied in various real-world scenarios where timely communication is essential.

It is useful in domains such as:

• **Banking Systems** – for transaction alerts, OTP verification, and security notifications

• **E-commerce Platforms** – for order confirmation, delivery updates, and promotional messages

• **Educational Institutions** – for announcements, exam results, and notifications

• **Healthcare Systems** – for appointment reminders and medical alerts

• **Business Applications** – for system alerts, updates, and internal communication

The system ensures that important information is delivered to users quickly and efficiently, improving overall communication.

3.6 Security Scope

Security is a critical aspect of the notification system, as it handles sensitive user data and important messages. The system implements security measures to protect data and ensure safe communication.

Key security aspects include:

• Secure handling of user information
• Controlled access to system functionalities
• Safe integration with external APIs
• Prevention of unauthorized access

3.7 Performance Scope

The system is designed to provide efficient performance under normal operating conditions. It ensures fast processing and delivery of notifications without significant delays.

Performance features include:
• Quick processing of notification requests
• Efficient handling of multiple users
• Optimized database operations using Hibernate
• Reduced response time

The system is capable of handling moderate workloads effectively, making it suitable for academic and small-scale applications.

3.8 Future Scope

The project provides a strong foundation that can be extended with advanced features in the future.

Possible enhancements include:

• Integration with **WhatsApp notifications**
• Implementation of **real-time notification systems**
• Use of **AI for smart notification routing**
• Development of a **mobile application interface**
• Addition of **notification scheduling and prioritization**

These improvements can make the system more powerful and suitable for large-scale applications.

3.9 Technical Limitations

While the system is efficient, it has certain limitations:

• Dependency on internet connectivity
• Reliance on third-party APIs for notification delivery

• Limited scalability for very large systems
• Possible delays due to external service providers

These limitations can be addressed in future versions of the system.

3.10 Performance Scope

The system is designed to handle moderate levels of user interaction efficiently. The use of optimized database queries ensures that messages are retrieved quickly. Additionally, the system avoids unnecessary data duplication, thereby improving performance.

3.11 Functional Scope in Detail

The functional scope of the system includes all operations related to user interaction and system behavior. The system allows users to perform multiple actions such as joining chat rooms, sending messages, uploading files, and deleting messages.

Each function is designed to ensure accuracy and efficiency. For example, message handling ensures that data is processed and displayed without delay.

3.12 Technical Limitations

While the system is designed to be efficient, it has certain technical limitations such as:

- Dependency on internet connectivity
- Limited scalability for large users
- No real-time communication

These limitations can be addressed in future versions.

3.13 Summary of Scope

The scope of the **Multi-Channel Notification Delivery System** covers all major aspects such as notification delivery, user management, data handling, and system performance.

The system provides a complete framework for understanding how modern notification systems operate. It ensures reliable communication, efficient processing, and secure data handling while allowing future enhancements.

Overall, the project offers a strong foundation for developing scalable and efficient notification platforms in real-world applications.

3.14 System Scalability Considerations

The Multi-Channel Notification Delivery System is designed with scalability in mind to handle an increasing number of users and notification requests. As modern applications grow, the system must be capable of processing large volumes of notifications efficiently without affecting performance.

The system uses a modular architecture, which allows different components such as notification processing, channel handling, and database operations to be managed independently. This design makes it easier to scale individual components based on system requirements.

By considering scalability, the system ensures that it can adapt to future growth and handle higher workloads, making it suitable for both academic and real-world applications.

CHAPTER 4

FEASIBILITY STUDY

4.1 Introduction to Feasibility Study

A feasibility study is an important step in system development that helps determine whether a project can be successfully implemented within the given constraints. It evaluates different aspects such as technical requirements, cost, usability, and time required for development.

For the **Multi-Channel Notification Delivery System**, the feasibility study ensures that the system can be developed efficiently using available resources and technologies. It also helps in identifying potential risks and challenges before implementation.

The feasibility study mainly focuses on:

• Technical feasibility
• Economic feasibility
• Operational feasibility
• Schedule feasibility

4.2 Technical Feasibility

Technical feasibility evaluates whether the required technology and tools are available to develop the system**.**

4.2.1 Availability of Technology

The system uses widely available technologies such as:

• Java
• Spring Boot
• Hibernate
• MySQL
• HTML, CSS, JavaScript

These technologies are:

• Easy to learn and implement
• Well-supported with documentation
• Suitable for academic projects

4.2.2 Development Complexity

The system follows a modular structure, which simplifies development. Each component such as notification processing, channel handling, and database management is developed separately. This reduces complexity and makes the system easier to maintain and extend.

This approach:

• Reduces development complexity
• Improves code readability
• Makes debugging easier
• Allows independent testing of modules

4.2.3 Integration Feasibility

The system requires integration with external services such as Email servers, SMS gateways, and Push notification providers.

This is feasible because:

• APIs are readily available
• Integration is supported by Spring Boot
• Standard protocols are used

4.2.4 Performance Feasibility

The system is capable of handling multiple notification requests efficiently.

Performance is ensured by:

• Optimized backend processing
• Efficient database queries using Hibernate
• Proper handling of request-response cycles

4.3 Economic Feasibility

Economic feasibility evaluates the cost-effectiveness of the project.

4.3.1 Development Cost

The system is economical because:

- Uses open-source technologies
- No licensing fees required
- Development tools are freely available

4.3.2 Operational Cost

Operational costs are minimal due to:

- Low hardware requirements
- Use of lightweight frameworks
- Basic hosting needs

4.3.3 Cost-Benefit Analysis

Benefits include:

- Learning experience
- Practical implementation
- Low cost with high value

4.4 Operational Feasibility

Operational feasibility evaluates how effectively the proposed system can be used in a real-world environment. It focuses on the system's usability, user acceptance, and its ability to perform required operations smoothly. For the Multi-Channel Notification Delivery System, operational feasibility ensures that the system is practical, user-friendly, and capable of meeting user requirements efficiently**.**

4.4.1 User Acceptance

User acceptance is an important factor in determining the success of any system. The proposed system is designed in a simple and understandable manner so that users can easily interact with it.
The system ensures that users receive notifications without needing any technical knowledge. Notifications are delivered automatically through available channels, making the process smooth and efficient.

The system is expected to be widely accepted because:

- It provides timely and accurate notifications
- It reduces communication delays
- It improves user engagement
- It simplifies the notification process

4.4.2 System Usability

System usability refers to how easy it is for users and administrators to interact with the system. The proposed system provides a clean and user-friendly interface that allows easy navigation and operation.

Key usability features include:

- Simple interface design
- Easy access to system functionalities
- Clear display of notification status
- Minimal user effort required

The system is designed to reduce complexity and ensure that all operations can be performed without confusion.

4.4.3 Administrative Control

The administrator plays a crucial role in managing the system. The system provides full control to the administrator to handle operations effectively.

Administrative features include:
- Managing user information
- Sending notifications to users
- Monitoring notification delivery status
- Viewing logs of system activities
- Controlling system operations

This ensures that the system functions smoothly and efficiently.

4.5 Schedule Feasibility

Schedule feasibility evaluates whether the proposed project can be completed within the given time frame. It ensures that all phases of system development, including planning, design, implementation, testing, and documentation, can be carried out efficiently within the available time.

For the **Multi-Channel Notification Delivery System**, the project is considered feasible in terms of schedule because it follows a well-defined and structured development process. The use of modern frameworks such as **Spring Boot** and **Hibernate** reduces development time by providing ready-made components and tools.

4.5.1 Project Planning

The initial phase involves understanding project requirements, defining objectives, and preparing a development plan. This phase ensures that the project starts with a clear direction and scope.

Activities include:

- Requirement analysis
- Identifying system features
- Preparing project timeline
- Resource allocation

4.5.2 Design Phase

In this phase, the system architecture and design are created. It includes designing modules, database structure, and system workflow.

Activities include:

- System architecture design
- Database design
- Module planning
- Interface design

4.5.3 Implementation Phase

This phase involves actual development of the system using selected technologies. Each module is developed separately and integrated into the system.

Activities include:

- Backend development using Spring Boot
- Database implementation using MySQL
- Integration of notification channels
- Coding of system functionalities

4.6 Legal Feasibility

Legal feasibility evaluates whether the proposed system complies with legal standards, data protection rules, and ethical practices. It ensures that the system does not violate any laws related to data usage, privacy, or communication.

For the **Multi-Channel Notification Delivery System**, legal feasibility is maintained by ensuring that user data is handled securely and responsibly. The system does not misuse personal information such as email addresses or phone numbers.

The system follows legal practices such as:

- Protecting user privacy
- Avoiding unauthorized use of data
- Ensuring secure communication
- Using trusted APIs for notification services

Additionally, the system avoids sending unauthorized or spam notifications, which ensures ethical use of communication channels.

4.7 Social Feasibility

Social feasibility refers to how well the system is accepted by users and its impact on society. It evaluates whether the system is beneficial and user-friendly.

The **Multi-Channel Notification Delivery System** improves communication by delivering timely alerts and updates. It helps users stay informed about important activities such as transactions, reminders, and system notifications.

The system contributes positively by:

- Enhancing communication efficiency
- Reducing delays in information delivery
- Supporting digital transformation
- Improving user satisfaction

The system is easy to use and does not require advanced technical knowledge, making it suitable for a wide range of users.

4.8 Resource Feasibility

Resource feasibility evaluates whether the required resources such as hardware, software, and human skills are available to develop the system.

Hardware Requirements

• Computer or laptop
• Internet connection

Software Requirements

• Java Development Kit (JDK)
• Spring Boot Framework
• Hibernate
• MySQL Database
• Web Browser

Human Resources

The project can be developed by individuals or small teams with knowledge of:
• Java programming
• Web development
• Database management

The availability of these resources ensures that the project can be developed without major difficulties.

4.9 Feasibility Conclusion

The feasibility study confirms that the **Multi-Channel Notification Delivery System** is practical and can be successfully developed using available resources and technologies.

The system is:
✓ Technically feasible
✓ Economically viable

✓ Operationally efficient
✓ Easy to implement and maintain

The project meets all requirements and can be completed within the given constraints. It provides a reliable and effective solution for notification delivery.

4.10 Risk Analysis

Risk analysis identifies potential issues that may arise during system development or operation and provides solutions to minimize them.

Possible risks include:
• Failure of notification delivery
• Dependency on third-party APIs
• Network connectivity issues
• Data security risks
• System performance issues

.

4.11 Summary

The feasibility study of the **Multi-Channel Notification Delivery System** demonstrates that the project is practical, efficient, and achievable within the given constraints. It evaluates various aspects such as technical, economic, operational, legal, and resource feasibility to ensure successful system development.

The study confirms that the required technologies are readily available and suitable for implementation. The system is cost-effective due to the use of open-source tools and requires minimal resources for development and operation. It is also user-friendly and can be easily accepted by users and administrators.

Additionally, the project can be completed within the academic timeline using a structured and modular development approach. Potential risks such as notification failure and dependency on external services are identified and can be minimized through proper design and implementation.

CHAPTER 5

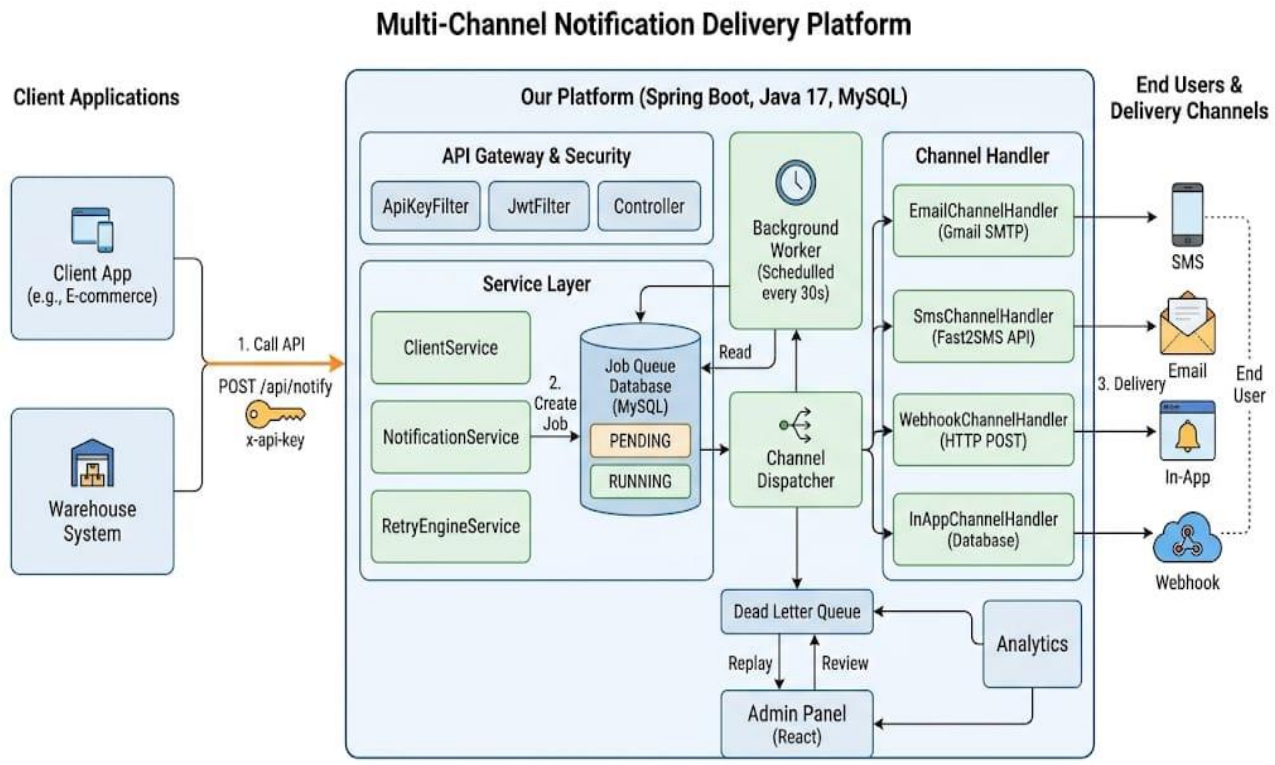
SYSTEM DESIGN

5.1 Introduction

System design is a crucial phase in software development that defines the architecture, components, modules, and data flow of the system. The **Multi-Channel Notification Delivery System** is designed using a modular and layered architecture to ensure scalability, maintainability, and reliability.

The system integrates notification processing, channel management, retry mechanisms, and logging into a unified platform. The design focuses on simplifying complex operations such as sending notifications through multiple channels while ensuring efficient and secure communication.

.



5.2 Architectural Design

The system follows a **three-tier architecture**, which includes:

1. Presentation Layer

This layer provides the user interface for interaction.

- HTML, CSS, JavaScript
- Displays notification status
- Takes user input

2. Business Logic Layer

Implemented using Spring Boot.

- Processes notification requests
- Selects communication channels
- Handles retry mechanism
- Manages system log

3. Data Layer

Managed using MySQL and Hibernate.

- Stores user details
- Stores notifications
- Maintains logs

5.3 Module Design

The Multi-Channel Notification Delivery System is designed using a modular approach, where the system is divided into different functional modules. Each module is responsible for performing a specific task, which improves system organization, maintainability, and scalability.

1. User Module

The User Module is responsible for managing user-related information required for sending notifications. It stores details such as user ID, email address, and phone number, which are used for communication.

This module ensures that user data is properly maintained and can be accessed whenever a notification needs to be sent. It also helps in associating users with specific notifications.

Functions:
• Store user details
• Manage contact information
• Retrieve user data for notification delivery

◈ 2. Notification Module

The Notification Module is the core component of the system. It handles the creation and processing of notifications.

This module receives notification requests, validates the input, and prepares the message for delivery. It ensures that the notification content is properly formatted before sending it to the channel module.

Functions:
• Create notification messages
• Process notification requests
• Validate input data
• Forward messages to channel

◈ 3. Access Control Mechanism

The Channel Module is responsible for handling different communication channels such as Email, SMS, and Push Notifications.

Each channel is managed separately, allowing the system to send notifications through multiple platforms. The module integrates with external APIs to deliver messages efficiently.

Functions:
• Send Email notifications
• Send SMS alerts
• Send Push notifications
• Manage channel configurations

◈ 4. Retry Module

The Retry Module ensures reliable notification delivery by handling failed messages. If a notification fails due to network issues or API errors, this module automatically retries sending the message.

The retry process is performed based on predefined rules, such as number of attempts and time intervals.

Functions:
• Detect failed notifications
• Retry message delivery
• Manage retry attempts
• Improve delivery success rate

◈ 5. Logging Module

The Logging Module maintains records of all system activities. It stores details of notifications, including delivery status, time, and channel used.

This module helps in monitoring system performance and identifying issues.

Functions:
• Store notification logs

• Track delivery status
• Record system activities
• Assist in debugging

5.4 **Data Flow Design**

The data flow design of the Multi-Channel Notification Delivery System describes how data moves through the system from input to output. It represents the flow of notification requests, processing, and delivery across different modules.

5.4.1 Data Flow Process

The overall data flow in the system is as follows:

1. The user or administrator sends a notification request
2. The system receives and validates the input data
3. The notification is processed by the Notification Module
4. The appropriate communication channel is selected
5. The message is sent through the selected channel
6. The delivery status is recorded in the database
7. If the notification fails, the Retry Module is triggered
8. The system updates the final status and logs the result

5.5 Security Design

Security is a critical aspect of the **Multi-Channel Notification Delivery System**, as it deals with sensitive user information such as email addresses, phone numbers, and important notifications like OTPs and alerts. The system is designed with multiple layers of security to ensure data protection, secure communication, and controlled access.

The security design focuses on preventing unauthorized access, protecting data during transmission, and maintaining the integrity of the system.

5.5.1 Authentication and Authorization

The system ensures that only authorized users and administrators can access and perform operations. Authentication verifies the identity of users, while authorization controls what actions they can perform.

Features include:

• Secure login mechanisms
• Role-based access control
• Restricted system operations

5.5.2 Secure API Communication

The system integrates with external services such as Email and SMS APIs. Secure communication is maintained between the system and these services.

Security measures include:

• Use of secure API endpoints
• Proper authentication keys
• Safe handling of API requests and responses

5.5.3 Data Protection

The system ensures that sensitive data is handled securely during storage and transmission.

Data protection methods include:

• Secure storage of user information
• Avoiding exposure of sensitive data
• Safe handling of notification content

5.5.4 Input Validation

Input validation ensures that only valid and safe data is processed by the system. This prevents common security threats such as injection attacks.

Validation includes:

• Checking input format

• Preventing invalid data
• Rejecting unauthorized inputs

5.5.5 Access Control Mechanism

The system implements access control to restrict system usage to authorized users only. Features include:

• Controlled access to modules
• Prevention of unauthorized operations
• Secure system usage

5.5.6 Protection Against Failures and Attacks

The system is designed to handle unexpected failures and protect against potential threats.

Security measures include:
• Error handling mechanisms
• Safe retry operations
• Protection from misuse of APIs

5.6 Database Design

The database design plays a crucial role in the **Multi-Channel Notification Delivery System**, as it is responsible for storing and managing all the data required for the system to function effectively. The system uses a relational database (**MySQL**) along with **Hibernate ORM** to handle data operations efficiently.

The database is designed to ensure data consistency, integrity, and fast retrieval. It stores user details, notification data, channel information, and delivery logs in a structured manner.

5.6.1 Tables Used in the System

The system consists of the following main tables:

• User Table
• Notification Table

• Channel Table
• Log Table

Each table serves a specific purpose in managing system data.

5.6.2 User Table

Purpose:

Stores details of users who receive notifications.

Attributes:

• id (Primary Key)
• name
• email
• phone number

Explanation:

Each user is uniquely identified using an ID. This table helps in managing user information and associating users with notifications.

5.6.3 Notification Table

Purpose:

Stores all notification messages sent by the system.

Attributes:

• id (Primary Key)
• message content
• status
• user_id (Foreign Key)

Explanation:

This table contains the notification content and its delivery status. It is linked to the User table through a foreign key.

5.7 Special Feature: Multi-Channel Notification

The system includes a special feature that supports multiple communication channels for delivering notifications:

"Multi-Channel Notification Delivery"

Rules:

• Notifications can be sent through Email, SMS, and Push Notifications
• Multiple channels can be used simultaneously
• If one channel fails, another channel can be used
• Ensures higher delivery success rat

5.8 System Workflow

1. Admin/User creates notification request

2. System processes the request

3. Appropriate channel is selected automatically

4. Notification is sent through selected channel(s)

5. Delivery status is stored in database

6. Retry mechanism is triggered if delivery fails

5.9 Detailed Working of AES Encryption Algorithm

The system uses a structured process to handle notification delivery efficiently.

Working Process

- Notification request is received
- Input data is validated
- Message is formatted
- Channel is selected
- Notification is sent through API
- Status is updated in database

Failure Handling:

- If notification fails
- Retry mechanism is triggered
- System attempts to resend message
- Final status is recorded

5.10 Advantages of AES in This Project

• High reliability in message delivery
• Faster communication
• Reduced failure rate
• Supports multiple communication platforms
• Flexible and scalable system

5.11 Channel Selection Algorithm

The system uses a channel selection mechanism to determine how notifications are delivered.

Steps:

1. Receive notification request
2. Check available channels
3. Identify user contact details
4. Select one or more channels
5. Send notification

Example:
Selected Channels → Email + SMS

5.12 Retry Mechanism Algorithm

1. System sends notification
2. If success → process ends
3. If failure → retry triggered
4. Retry for limited attempts
5. If still failed → mark as failed

5.13 Data Flow Explanation

The system follows a structured data flow:

• Input from user/admin
• Processing in controller
• Business logic execution
• Channel API interaction
• Database update
• Response sent to UI

5.14 Detailed System Workflow Analysis

The **Multi-Channel Notification Delivery System** follows a structured workflow that ensures reliable communication between the system and users while maintaining efficiency and data integrity.

When a notification request is initiated, the system first validates the input data and creates a processing request. The backend system handles this request and determines the appropriate communication channel based on availability and configuration.

Once the channel is selected, the notification is sent using external APIs such as Email or SMS services. If multiple channels are configured, the system can send notifications through more than one channel simultaneously.

If the notification fails due to network issues or API errors, the retry mechanism is activated. The system attempts to resend the message until the maximum retry limit is reached.

The delivery status of each notification is stored in the database. This allows administrators to track whether the message was successfully delivered or failed.

This workflow ensures that the system provides reliable, efficient, and scalable notification delivery while maintaining simplicity and performance

s

5.15 Detailed Module Interaction

The system consists of multiple modules that interact with each other to perform notification delivery efficiently. These modules include the user module, notification module, channel module, retry module, and logging module.

The **User Module** manages user information required for communication. The **Notification Module** processes requests and prepares messages. The **Channel Module** interacts with external APIs to send notifications.

The **Retry Module** handles failed notifications and ensures re-delivery, while the **Logging Module** maintains records of all activities.

The interaction between these modules is managed through the controller layer, which acts as a bridge between frontend and backend. This modular design improves system efficiency and maintainability.

5.16 Detailed Security Architecture

Security is an important aspect of the **Multi-Channel Notification Delivery System**. The system implements multiple layers of security to protect user data and ensure safe communication.

The first layer is **authentication**, where only authorized users can access the system. The second layer is **authorization**, which controls system operations.

The third layer is **data security**, where sensitive information such as user details and notifications are handled securely.

Secure API communication ensures safe interaction with external services. Input validation prevents invalid or malicious data from entering the system.

These security measures ensure that the system remains safe, reliable, and protected from unauthorized access and data misuse.

5.17 Internal Working of Message Processing

The internal working of the **Multi-Channel Notification Delivery System** involves multiple steps to ensure that notifications are processed and delivered efficiently. When a notification request is

received, the system first performs input validation to check whether the provided data is correct and complete.

After validation, the notification message is processed and formatted according to the requirements of the selected communication channel. The system then determines the appropriate channel or channels for delivery based on availability and user details.

The notification is sent through external APIs such as Email or SMS services. Once the message is delivered, the system updates the delivery status in the database. If the notification fails, the retry mechanism is activated to resend the message.

This structured internal process ensures accurate processing, reliable delivery, and efficient handling of notifications.

5.18 Detailed Explanation of Access Control

Access control is implemented to restrict entry to chat rooms. Each room is assigned a unique Channel handling is a key component of the system that enables communication through multiple platforms. Each communication channel, such as Email, SMS, and Push Notification, is handled separately to ensure flexibility and efficiency.

The system integrates with third-party APIs to send notifications through these channels. Each channel has its own configuration, including authentication details and communication protocols.

When a notification request is processed, the system selects the appropriate channel based on user information and system configuration. The message is then sent through the selected channel, and the result is recorded.

If one channel fails, the system can switch to another available channel, ensuring higher reliability and successful delivery of notifications.

5.19 System Scalability Considerations

The **Multi-Channel Notification Delivery System** is designed to handle increasing workloads and user demands efficiently. Scalability is an important aspect that ensures the system can grow without affecting performance.

The system uses a modular architecture, which allows individual components such as notification processing and channel handling to be scaled independently. This makes it easier to handle a large number of notification requests.

The system integrates with third-party APIs to send notifications through these channels. Each channel has its own configuration, including authentication details and communication protocols.

When a notification request is processed, the system selects the appropriate channel based on user information and system configuration. The message is then sent through the selected channel, and the result is recorded.

Additionally, the system can be enhanced by integrating cloud-based services and load balancing techniques. This helps distribute workload and improve system performance.

By considering scalability, the system ensures that it remains efficient, reliable, and capable of supporting future growth.

CHAPTER 6

EXISTING SYSTEM

6.1 Introduction to Existing System

In today's digital environment, notification systems play an essential role in communication between applications and users. Many existing systems rely on sending notifications through a single communication channel such as Email or SMS. These systems are widely used in applications like banking alerts, OTP verification, and promotional messaging

.

Although these systems provide basic functionality, they are not always efficient or reliable for modern applications. As user expectations increase, there is a need for more advanced notification mechanisms that can handle multiple communication channels effectively.

6.2 Description of Existing Systems

Existing notification systems typically operate using a single-channel approach. These systems are designed to send notifications through one specific communication medium, such as Email or SMS.

The architecture of these systems often includes:

• Integration with a single API (Email or SMS)
• Basic message sending functionality
• Limited error handling mechanisms
• Simple database storage

These systems are suitable for small-scale applications but may not perform well in complex environments where reliability and scalability are required.

6.3 Features of Existing Systems

Existing notification systems provide the following features:

• Sending notifications through Email or SMS
• Basic message delivery functionality
• Simple user notification handling
• Limited logging capabilities
Some advanced systems may also include:
• Scheduled notifications
• Basic delivery status tracking

However, these features are often limited and not sufficient for modern applications.

6.4 Limitations of Existing Systems

Despite their advanced features, existing systems have several limitations:

1.Single Channel Dependency

Notifications are sent through only one channel, making the system unreliable if that channel fails.

2. Lack of Reliability

Most applications are closed-source and do not allow modification according to user requirements.

3. High Resource Usage

They require:

- High bandwidth
- Advanced servers
- Large storage capacity

4. Lack of Centralized Management

There is no single platform to manage all notifications, making monitoring and control difficult.

5. Security Complexity

While they provide encryption, the implementation of these systems is complex and not easily understandable for academic purposes.

6.5 Need for Proposed System

Due to the limitations of existing systems, there is a need for a better solution that can provide reliable and efficient notification delivery.

The proposed system aims to:

• Provide multi-channel notification delivery
• Improve reliability using retry mechanisms
• Offer centralized management of notifications
• Simplify system design and implementation

6.6 Summary

The existing notification systems, although useful, have several limitations in terms of reliability, scalability, and flexibility. They are not suitable for modern applications that require efficient and reliable communication.

CHAPTER 7

PROPOSED APPLICATION

7.1 Introduction to Proposed System

The proposed system is a **Multi-Channel Notification Delivery System** designed to overcome the limitations of existing single-channel notification systems. It provides a centralized platform for sending notifications through multiple communication channels such as Email, SMS, and Push Notifications.

The system is developed using modern technologies like **Spring Boot** and **Hibernate**, ensuring efficient processing, scalability, and maintainability. The design focuses on improving reliability, reducing message failure, and enhancing communication between systems and users.

7.2 Overview of Proposed System

The system provides a unified interface for sending notifications. Instead of integrating multiple APIs separately, the system acts as an intermediary layer that handles all communication channels.

The system allows users or administrators to:
• Send notifications through multiple channels
• Track delivery status
• Manage users and notification data
• Handle failures using retry mechanisms.

This approach simplifies system design and improves overall efficiency.

7.3 Key Features of Proposed System

1.Multi-Channel Notification Delivery

The system supports sending notifications through multiple channels such as Email, SMS, and Push Notifications. This ensures higher reliability and better communication.

2. Centralized Notification Management

All notification operations are handled in a single platform, making it easy to manage and monitor system activities.

3. Retry Mechanism

The system automatically retries failed notifications to ensure successful delivery.

4. Notification Logging and Tracking

The system maintains logs of all notifications, including delivery status and timestamps.

5. Scalable Architecture

The system is designed to handle increasing workloads and can be extended to support additional features.

6. Easy Integration

The system can be easily integrated with external APIs and other applications.

7.4 Advantages of Proposed System

The proposed system offers several advantages over existing systems:

• Improved Reliability – Multi-channel delivery reduces failure rate

• Better Performance – Efficient processing ensures fast delivery

• Centralized Control – Easy management of notifications

• Scalability – Supports growing number of users

• Flexibility – Can be extended with new features

7.5 Working of Proposed System

The system follows a structured process for notification delivery:

1. User/Admin creates notification request
2. System processes the request
3. Appropriate channel is selected
4. Notification is sent through API
5. Status is stored in database
6. Retry mechanism handles failures

This workflow ensures reliable and efficient communication

7.6 Efficiency of Proposed System

The system is designed to be efficient in terms of performance and resource utilization.

Efficiency features include:

• Fast processing of notification requests
• Reduced system complexity

• Optimized database operations
• Minimal delay in message delivery

7.7 Reliability of the System

Reliability is ensured through:

• Multi-channel communication
• Retry mechanism
• Error handling
• Logging and monitoring

These features ensure that notifications are delivered successfully even in case of failures.

.

7.8 Scalability of the System

The system is designed to handle increasing workloads and can be scaled easily.

Scalability features include:
• Modular architecture
• Independent components
• Support for additional channels

7.9 Security of the System

The system ensures secure communication by:

• Protecting user data
• Using secure APIs
• Implementing access control
• Validating inputs

7.10 Future Enhancements

The system can be extended with advanced features such as:

• WhatsApp notifications
• AI-based notification routing
• Real-time push notifications
• Mobile application support
• Advanced analytics

7.8 Summary

The **Multi-Channel Notification Delivery System** provides a reliable, efficient, and scalable solution for modern communication needs. It overcomes the limitations of existing systems by offering multi-channel delivery, centralized management, and advanced error handling mechanisms.

The system serves as a strong foundation for future improvements and real-world applications.

CHAPTER 8

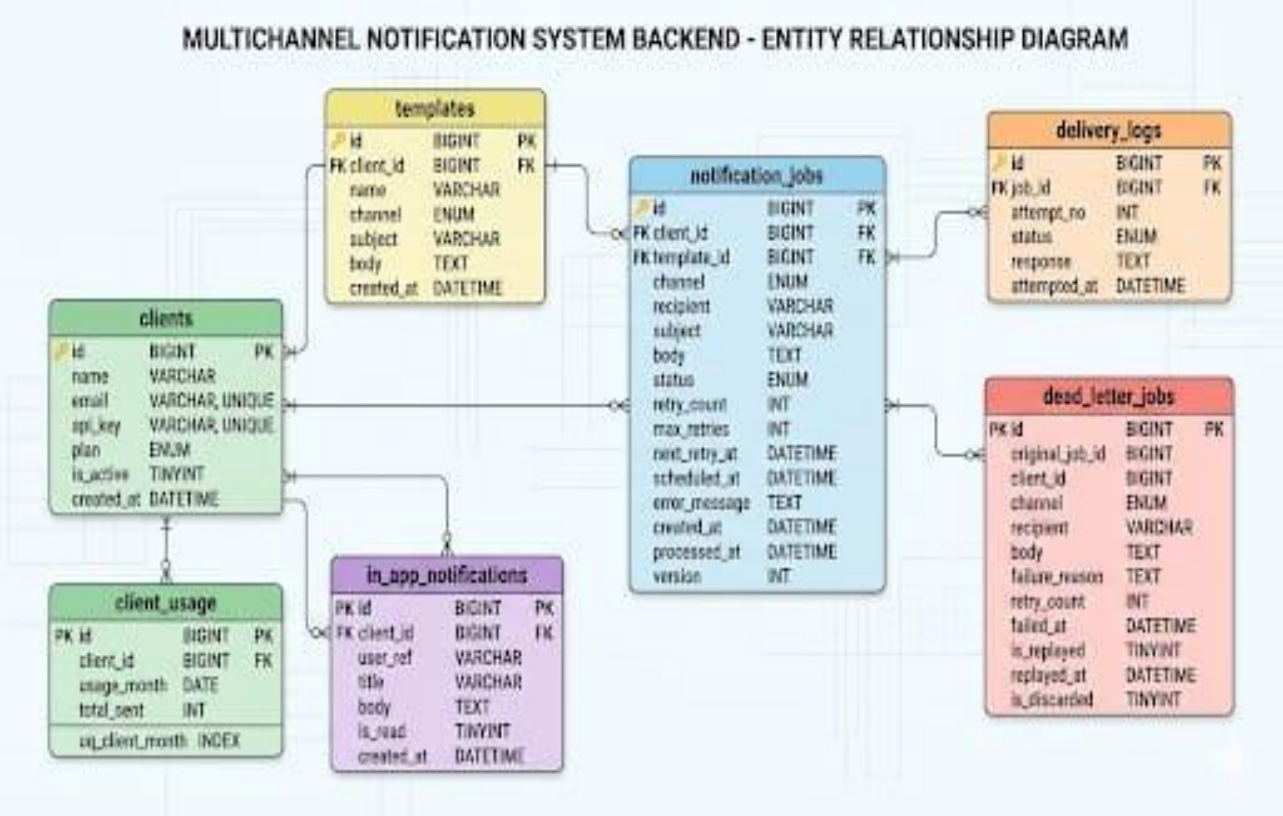
ER DIAGRAM

8.1 Introduction to ER Diagram

The **Entity Relationship (ER) Diagram** is a graphical representation of the database structure of the system. It shows how different entities are related to each other and how data flows between them.

In the **Multi-Channel Notification Delivery System**, the ER diagram helps in understanding how user data, notification data, channel information, and logs are organized and connected. It provides a clear view of the database design and ensures proper data management.

The ER diagram plays an important role in designing a structured and efficient database system.



8.2 Entities in the System

1. User

This entity represents the users who receive notifications.

Attributes:
• id (Primary Key)
• name
• email
• phone number

2. Notification

This entity represents the messages sent by the system.

Attributes:
• id (Primary Key)
• message
• status
• created_at
• user_id (Foreign Key**)**

3. Channel

This entity represents different communication channels used for sending notifications.

Attributes:
• id (Primary Key)
• channel_type (Email / SMS / Push)

4. Log

This entity stores delivery status of notifications.

Attributes:
• id (Primary Key)
• notification_id (Foreign Key)
• status
• timestamp

8.3 Relationships

The relationships between entities are defined as follows:

1.User → Notification

• One user can receive multiple notifications
• Relationship type: One-to-Many

2. Notification → Log

• One notification can have multiple log entries
• Relationship type: One-to-Many

3.Notification → Channel

• One notification can be sent through multiple channels
• Relationship type: Many-to-Many

8.4 ER Diagram Explanation

The ER diagram shows how the system manages data efficiently. The **User entity** stores user information, while the **Notification entity** stores messages sent to users.
Each notification is linked to a user and can be delivered through one or more channels. The **Channel entity** represents different communication methods, and the **Log entity** keeps track of delivery status.

This structure ensures that data is organized, consistent, and easy to manage.

8.5 Database Logic

The database logic ensures proper data handling and relationships.

Key aspects include:

- Use of primary keys for unique identification
- Use of foreign keys to maintain relationships
- Avoidance of data redundancy
- Efficient data retrieval

8.6 Role of ER Diagram

The ER diagram is important because:

- It helps in designing the database structure
- Ensures proper relationships between entities
- Improves system efficiency
- Reduces errors in database design

8.7 Normalization

Normalization is used to organize data efficiently and reduce redundancy.

Normalization is a database design technique used to organize data efficiently by reducing redundancy and improving data integrity. It involves dividing large tables into smaller, well-structured tables and establishing relationships between them.

In the **Multi-Channel Notification Delivery System**, normalization is applied to ensure that data is stored in a structured and efficient manner. It helps in eliminating duplicate data, maintaining consistency, and improving overall system performance.

Benefits include:
• Eliminates duplicate data
• Improves data consistency
• Enhances database performance

This prevents data inconsistency and ensures structured storage.

8.8 Summary

The ER diagram of the **Multi-Channel Notification Delivery System** provides a clear representation of the database structure. It ensures efficient data organization, proper relationships, and smooth system operation.

The design supports scalability and future enhancements, making it suitable for both academic and real-world applications.

CHAPTER 9

DATABASE SCREENSHOTS AND EXPLANATION

9.1 Introduction to Database Design

The database is an essential component of the **Multi-Channel Notification Delivery System**, as it stores and manages all the data required for system operations. The system uses a relational database (**MySQL**) along with **Hibernate ORM** to ensure efficient data handling and interaction.

The database is designed to maintain:

• Data integrity
• Efficient storage
• Fast retrieval
• Secure data handling

The structure of the database ensures that all system components can interact smoothly and perform operations without data inconsistency.

9.2 Database Tables Overview

The system consists of the following main tables:

1. User Table
2. Notification Table
3. Channel Table
4. Log Table

Each table is designed to perform a specific role in the system.

```
CREATE DATABASE notification_platform;

USE notification_platform;

SHOW TABLES;

SELECT * FROM clients;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Tables_in_notification_platform
client_usage
clients
dead_letter_jobs
delivery_logs
flyway_schema_history
in_app_notifications
notification_jobs
templates

Result 1

Output

Action Output

#	Time	Action	Message
1	15:19:41	USE notification_platform	0 row(s) affected
2	15:20:25	SHOW TABLES	8 row(s) returned

```
SELECT * FROM clients;

SELECT * FROM notification_jobs;
SELECT * FROM clients;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	name	email	api_key	plan	is_active	created_at
1	Test Company	testcompany@gmail.com	35ae6504d0094d428e8ee155b5ab2122	FREE	1	2026-04-08 10:54:43
2	shopEasy	shopeasy@gmail.com	697c1c0f8ac149adbb2d42f74c7cb118	FREE	1	2026-04-09 21:28:09
3	Zomato	zomato@gmail.com	7d84e662a99845b1a9352f7f4c6ee1b4	FREE	1	2026-04-10 09:42:19
4	flipkart	flipkart@gmail.com	3c5850e52a2f4c51b6b62a4b166c7e77	FREE	1	2026-04-10 10:34:13
5	swiggy	swiggy@gmail.com	5fce4dcc1b04430889e7d77ff58c0004	FREE	1	2026-04-10 10:39:01
6	ShopEasy	vivekgkuliyadi2004@gmail.com	b1671d42c3fb46838390d105a0fc1f76	FREE	1	2026-04-10 11:25:15
7	abcd	abcd@gmail.com	567d0e8bf91c41ebac569a96437036ce	FREE	1	2026-04-12 20:11:36
8	TestCorp	testcorp@gmail.com	713ee6068504431ba2aaca9ae6c0315d	FREE	1	2026-04-13 00:41:13
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
9  SELECT * FROM notification_jobs;
10 SELECT * FROM clients;
11 SELECT * FROM delivery_logs;
12 SELECT * FROM templates;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	client_id	template_id	channel	recipient	subject	body	status	retry_count	max_retrie
1	3	NULL	EMAIL	builproject55@gmail.com	Order Confirmed	Hi Vivek, your order ORD-441 worth Rs.599 is c...	FAILED	4	4
2	3	NULL	EMAIL	buildproject55@gmail.com	Order Confirmed	Hi Vivek, your order ORD-441 worth Rs.599 is c...	COMPLETED	3	4
3	3	NULL	EMAIL	buildproject55@gmail.com	Order Confirmed	Hi Vivek, your order ORD-441 worth Rs.599 is c...	COMPLETED	3	4
4	4	NULL	EMAIL	builproject55@gmail.com	Order Confirmed	Hi Vivek, your order ORD-441 worth Rs.599 is c...	COMPLETED	3	4
5	4	NULL	EMAIL	buildproject55@gmail.com	Order Confirmed	Hi Vivek, your order ORD-441 worth Rs.599 is c...	COMPLETED	2	4
6	4	NULL	EMAIL	buildproject55@gmail.com	Order Confirmed	Hi Vivek, your order ORD-441 worth Rs.599 is c...	COMPLETED	0	4
7	4	NULL	IN_APP	user101	Order Shipped	Your order ORD-441 is out for delivery	COMPLETED	0	4
8	4	NULL	WEBHOOK	https://webhook.site/bd566edb-d49b-493e-b2...	NULL	{"event": "order.confirmed", "orderId": "ORD-4...	COMPLETED	0	4
9	4	NULL	SMS	7483925785	NULL	Hi Vivek, your order ORD-441 worth Rs.599 is c...	FAILED	4	4
10	4	NULL	SMS	9481663929	NULL	Hi Vivek, your order ORD-441 worth Rs.599 is c...	FAILED	4	4
11	4	NULL	SMS	9481663929	NULL	Hi Vivek, your order ORD-441 worth Rs.599 is c...	FAILED	4	4
12	8	NULL	EMAIL	vivekg1234umbal@gmail.com	Testing the proj...	hi this is vivek and im testing the project. please...	COMPLETED	0	4
13	8	NULL	IN_APP	user123		this is testing for inapp channel	COMPLETED	0	4
14	8	NULL	EMAIL	abcd@gmail.com	hi	is this sent?	COMPLETED	0	4
15	8	NULL	EMAIL	vivekg1234umbal@gmail.com	checking	is this went?	COMPLETED	0	4
16	8	NULL	IN_APP	vivek345		checking the message	COMPLETED	0	4

```
11 SELECT * FROM delivery_logs;
12 SELECT * FROM templates;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	job_id	attempt_no	status	response	attempted_at
1	12	1	SUCCESS	Delivered successfully	2026-04-13 00:47:54
2	13	1	SUCCESS	Delivered successfully	2026-04-13 00:51:24
3	14	1	SUCCESS	Delivered successfully	2026-04-13 00:59:59
4	15	1	SUCCESS	Delivered successfully	2026-04-13 19:11:51
5	16	1	SUCCESS	Delivered successfully	2026-04-13 19:24:52
6	17	1	SUCCESS	Delivered successfully	2026-04-14 00:17:44
NULL	NULL	NULL	NULL	NULL	NULL

9.3 User Table

Purpose

The User table stores information about users who receive notifications.

Attributes

• id (Primary Key)
• name
• email
• phone number

Explanation

Each user is uniquely identified by an ID. This table is used to store contact details required for sending notifications through different channels.

9.4 Notification Table

Purpose

The Notification table stores all messages sent by the system.

Attributes

• id (Primary Key)
• message
• status
• created_at
• user_id (Foreign Key)

Explanation

This table stores the content of notifications and their delivery status. It is linked to the User table to identify the recipient.

9.5 Channel Table

Purpose

The Channel table stores information about different communication channels.

Attributes

• id (Primary Key)
• channel_type (Email, SMS, Push)

Explanation
This table helps in identifying the communication method used for sending notifications

9.6 Log Table

Purpose

The Log table stores delivery details of notifications.
Attributes

• id (Primary Key)
• notification_id (Foreign Key)
• status
• timestamp

Explanation

This table tracks whether a notification was successfully delivered or failed. It helps in monitoring system performance

```
CREATE TABLE clients (
        id        BIGINT AUTO_INCREMENT PRIMARY KEY,
        name      VARCHAR(100) NOT NULL,
        email     VARCHAR(100) NOT NULL UNIQUE,
        api_key   VARCHAR(64)  NOT NULL UNIQUE,
        plan      ENUM('FREE', 'PAID') DEFAULT 'FREE',
        is_active TINYINT(1)   DEFAULT 1,
        created_at DATETIME    NOT NULL,
        INDEX idx_api_key (api_key)
);

CREATE TABLE templates (
        id        BIGINT AUTO_INCREMENT PRIMARY KEY,
        client_id BIGINT       NOT NULL,
        name      VARCHAR(100) NOT NULL,
        channel   ENUM('EMAIL', 'SMS', 'IN_APP', 'WEBHOOK'),
        subject   VARCHAR(255),
        body      TEXT         NOT NULL,
        created_at DATETIME    NOT NULL,
        FOREIGN KEY (client_id) REFERENCES clients(id)
```

```
);

CREATE TABLE notification_jobs (
        id          BIGINT AUTO_INCREMENT PRIMARY KEY,
        client_id   BIGINT      NOT NULL,
        template_id BIGINT,
        channel     ENUM('EMAIL', 'SMS', 'IN_APP', 'WEBHOOK'),
        recipient   VARCHAR(255) NOT NULL,
        subject     VARCHAR(255),
        body        TEXT        NOT NULL,
        status  ENUM(‘PENDING’,’RUNNING’,’COMPLETED’,
        ‘RETRYING’,’FAILED’)DEFAULT ‘PENDING’,
        retry_count   INT         DEFAULT 0,
        max_retries   INT         DEFAULT 4,
        next_retry_at DATETIME,
        scheduled_at  DATETIME,
        error_message TEXT,
        created_at    DATETIME    NOT NULL,
        processed_at  DATETIME,
        version     INT         DEFAULT 0,
```

```
                    FOREIGN KEY (client_id) REFERENCES clients(id),
                    INDEX idx_status_retry (status, next_retry_at),
                    INDEX idx_client_status (client_id, status)
);

CREATE TABLE dead_letter_jobs (
                    id              BIGINT AUTO_INCREMENT PRIMARY KEY,
                    original_job_id BIGINT    NOT NULL,
                    client_id       BIGINT    NOT NULL,
                    channel         ENUM('EMAIL', 'SMS', 'IN_APP', 'WEBHOOK'),
                    recipient       VARCHAR(255),
                    body            TEXT,
                    failure_reason  TEXT,
                    retry_count     INT,
                    failed_at       DATETIME  NOT NULL,
                    is_replayed     TINYINT(1) DEFAULT 0,
                    replayed_at     DATETIME,
                    is_discarded    TINYINT(1) DEFAULT 0
);
```

```
CREATE TABLE delivery_logs (
        id          BIGINT AUTO_INCREMENT PRIMARY KEY,
        job_id      BIGINT    NOT NULL,
        attempt_no  INT       NOT NULL,
        status      ENUM('SUCCESS', 'FAILURE'),
        response    TEXT,
        attempted_at DATETIME  NOT NULL,
        FOREIGN KEY (job_id) REFERENCES notification_jobs(id),
        INDEX idx_job_id (job_id)
);


CREATE TABLE in_app_notifications (
        id          BIGINT AUTO_INCREMENT PRIMARY KEY,
        client_id   BIGINT       NOT NULL,
        user_ref    VARCHAR(100) NOT NULL,
        title       VARCHAR(255),
        body        TEXT,
        is_read     TINYINT(1)   DEFAULT 0,
        created_at DATETIME     NOT NULL,
        INDEX idx_user_read (client_id, user_ref, is_read)
```

```
);


CREATE TABLE client_usage (
        id          BIGINT AUTO_INCREMENT PRIMARY KEY,
        client_id   BIGINT NOT NULL,
        usage_month DATE   NOT NULL,
        total_sent  INT    DEFAULT 0,
        UNIQUE KEY uq_client_month (client_id, usage_month)
);
```

CHAPTER 10

CODING AND APPLICATION SCREENSHOTS

10.1 Introduction

This chapter describes the implementation of the **Multi-Channel Notification Delivery System**, including backend development, frontend interface, and system functionalities. It explains how the system is built using technologies such as **Spring Boot, Hibernate, and MySQL**, and how different components work together to deliver notifications.

The chapter also includes application screenshots to demonstrate the working of the system.

10.2 Backend Implementation

The backend of the system is developed using **Spring Boot**, which follows the **MVC (Model-View-Controller)** architecture. It handles all business logic, request processing, and integration with external APIs.

Features of Backend:

• Processes notification requests
• Integrates with Email and SMS APIs
• Handles retry mechanisms
• Manages database operations using Hibernate

ChannelDispatcher.java

```
package com.vivek.notification.notification_platform.channel;

import com.vivek.notification.notification_platform.entity.NotificationJob;
```

```
import lombok.RequiredArgsConstructor;

import org.springframework.stereotype.Component;

import java.util.List;


@Component

@RequiredArgsConstructor

public class ChannelDispatcher {

    private final List<ChannelHandler> handlers;

    public void dispatch(NotificationJob job) throws Exception {

        String channel = job.getChannel().name();

        ChannelHandler handler = handlers.stream()

                .filter(h -> h.getChannel().equals(channel))

                .findFirst()

                .orElseThrow(() -> new RuntimeException("No handler found for channel:
" + channel));

        handler.deliver(job);

    }

}
```

ChannelHandler.java

```
package com.vivek.notification.notification_platform.channel;
```

```
import com.vivek.notification.notification_platform.entity.NotificationJob;
public interface ChannelHandler {
    void deliver(NotificationJob job) throws Exception;
    String getChannel();
}
```

EmailChannelHandler.java

```
package com.vivek.notification.notification_platform.channel;
import com.vivek.notification.notification_platform.entity.NotificationJob;
import lombok.RequiredArgsConstructor;
import org.springframework.mail.javamail.JavaMailSender;
import org.springframework.mail.javamail.MimeMessageHelper;
import org.springframework.stereotype.Component;
import jakarta.mail.internet.MimeMessage;

@Component
@RequiredArgsConstructor
public class EmailChannelHandler implements ChannelHandler {
 private final JavaMailSender mailSender;
    @Override
```

```
    public void deliver(NotificationJob job) throws Exception {
        MimeMessage message = mailSender.createMimeMessage();
        MimeMessageHelper helper = new MimeMessageHelper(message, true);

        helper.setTo(job.getRecipient());
        helper.setSubject(job.getSubject() != null ? job.getSubject() : "Notification");
        helper.setText(job.getBody(), true);

        mailSender.send(message);
    }
    @Override
    public String getChannel() {
        return "EMAIL";
    }
}
```

InAppChannelHandler.java

```
package com.vivek.notification.notification_platform.channel;
import com.vivek.notification.notification_platform.entity.InAppNotification;
```

```
import com.vivek.notification.notification_platform.entity.NotificationJob;
import com.vivek.notification.notification_platform.repository.InAppNotification
Repository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Component;
import java.time.LocalDateTime;

@Component
@RequiredArgsConstructor
public class InAppChannelHandler implements ChannelHandler {
    private final InAppNotificationRepository inAppNotificationRepository;
    @Override
    public void deliver(NotificationJob job) throws Exception {
        InAppNotification notification = new InAppNotification();
        notification.setClient(job.getClient());
        notification.setUserRef(job.getRecipient());
        notification.setTitle(job.getSubject());
        notification.setBody(job.getBody());
        notification.setRead(false);
        notification.setCreatedAt(LocalDateTime.now());
```

```
        inAppNotificationRepository.save(notification);
    }
    @Override
    public String getChannel() {
        return "IN_APP";
    }
}
```

SmsChannelHandler.java

```
package com.vivek.notification.notification_platform.channel;
import com.vivek.notification.notification_platform.entity.NotificationJob;
import lombok.RequiredArgsConstructor;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestTemplate;
import org.springframework.http.*;
import java.util.HashMap;
import java.util.Map;
@Component
@RequiredArgsConstructor
```

```
public class SmsChannelHandler implements ChannelHandler {
    @Value("${app.sms.api.key}")
    private String smsApiKey;

    @Override
    public void deliver(NotificationJob job) throws Exception {
        RestTemplate restTemplate = new RestTemplate();

        HttpHeaders headers = new HttpHeaders();
        headers.setContentType(MediaType.APPLICATION_JSON);
        headers.set("authorization", smsApiKey);

        Map<String, Object> body = new HashMap<>();
        body.put("sender_id", "NOTIFY");
        body.put("message", job.getBody());
        body.put("language", "english");
        body.put("route", "qt");
        body.put("numbers", job.getRecipient());
        HttpEntity<Map<String, Object>> request = new HttpEntity<>(body, headers);
        ResponseEntity<String> response = restTemplate.postForEntity(
```

```
            "https://www.fast2sms.com/dev/bulkV2",
            request,
            String.class
        );
        if (!response.getStatusCode().is2xxSuccessful()) {
            throw new RuntimeException("SMS delivery failed: " + response.getBody());
        }
    }

    @Override
    public String getChannel() {
        return "SMS";
    }
}
```

WebhookChannelHandler.java

```
package com.vivek.notification.notification_platform.channel;

import com.vivek.notification.notification_platform.entity.NotificationJob;
```

```
import lombok.RequiredArgsConstructor;

import org.springframework.http.*;

import org.springframework.stereotype.Component;

import org.springframework.web.client.RestTemplate;

@Component

@RequiredArgsConstructor

public class WebhookChannelHandler implements ChannelHandler {

    @Override

    public void deliver(NotificationJob job) throws Exception {

        RestTemplate restTemplate = new RestTemplate();

        HttpHeaders headers = new HttpHeaders();

        headers.setContentType(MediaType.APPLICATION_JSON);

        HttpEntity<String> request = new HttpEntity<>(job.getBody(), headers);

        ResponseEntity<String> response = restTemplate.postForEntity(

                job.getRecipient(),

                request,

                String.class

        );

        if (!response.getStatusCode().is2xxSuccessful()) {

            throw new RuntimeException("Webhook delivery failed: " +
response.getBody());
```

```
        }
    }
 @Override
 public String getChannel() {
        return "WEBHOOK";
    }
}
```

NotificationWorker.java

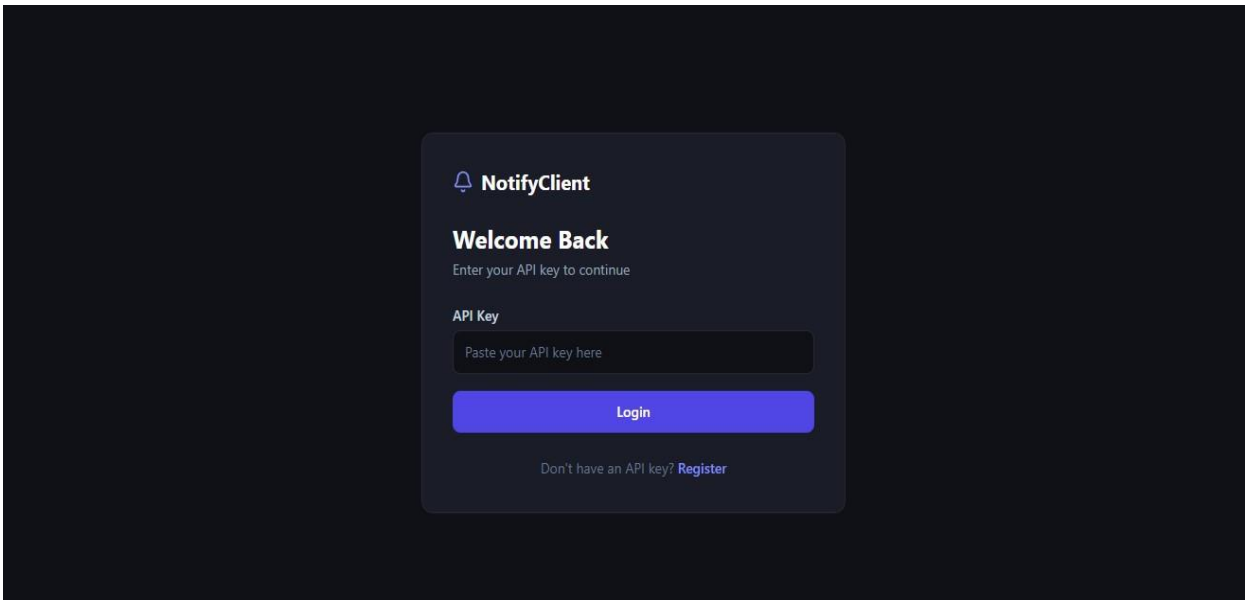
```
package com.vivek.notification.notification_platform.worker;
import com.vivek.notification.notification_platform.channel.ChannelDispatcher;
import com.vivek.notification.notification_platform.entity.DeliveryLog;
import com.vivek.notification.notification_platform.entity.NotificationJob;
import com.vivek.notification.notification_platform.repository.DeliveryLog
Repository;
import com.vivek.notification.notification_platform.repository.NotificationJob
Repository;
import com.vivek.notification.notification_platform.service.RetryEngineService;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
```

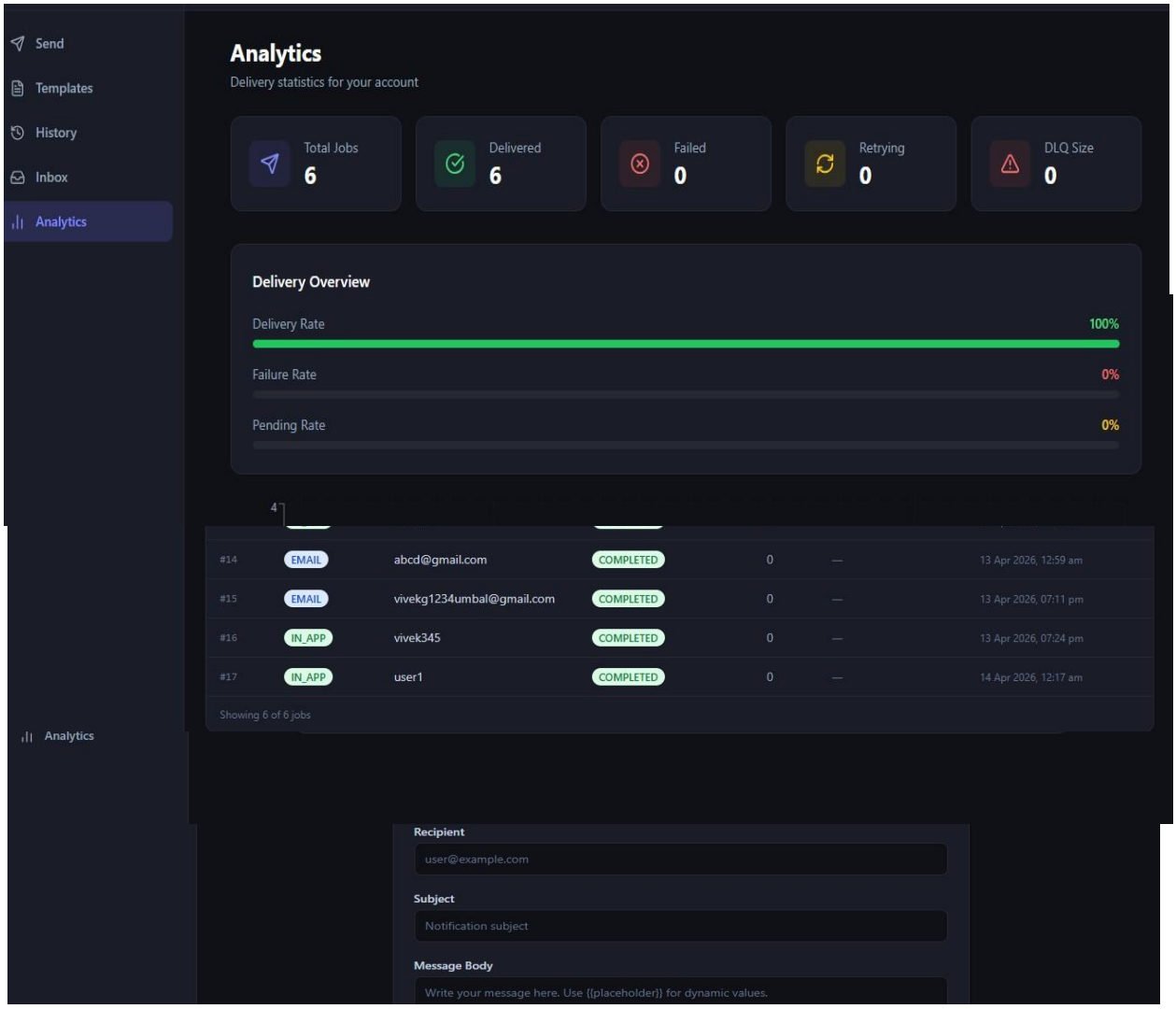
```
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;
import java.time.LocalDateTime;
import java.util.List;

@Component
@RequiredArgsConstructor
@Slf4j
public class NotificationWorker {
    private final NotificationJobRepository jobRepository;
    private final DeliveryLogRepository deliveryLogRepository;
    private final ChannelDispatcher channelDispatcher;
    private final RetryEngineService retryEngineService;

    @Scheduled(fixedDelay = 30000) // runs every 30 seconds
    @Transactional
    public void processJobs() {
        log.info("Worker started - checking for pending jobs");
        List<NotificationJob> jobs = jobRepository
```

```
            .findJobsReadyToProcess(LocalDateTime.now());
        if (jobs.isEmpty()) {
            log.info("No jobs to process");
            return;
        }


        log.info("Found {} jobs to process", jobs.size());
        for (NotificationJob job : jobs) {
            try {
                processJob(job);
          } catch (org.springframework.orm.ObjectOptimisticLockingFailureException e)
{
                log.info("Job {} already picked up by another thread - skipping",
job.getId());
            }
        }
    }
    private void processJob(NotificationJob job) {
        // Mark as RUNNING
        job.setStatus(NotificationJob.JobStatus.RUNNING);
        jobRepository.save(job);
```

```
        try {
            // Try to deliver
            channelDispatcher.dispatch(job);


            // Success - log it
            logAttempt(job, DeliveryLog.DeliveryStatus.SUCCESS, "Delivered
successfully");
            // Mark as COMPLETED
            retryEngineService.handleSuccess(job);
            log.info("Job {} delivered successfully via {}",
                    job.getId(), job.getChannel());
        } catch (Exception e) {
            // Failed - log it
            logAttempt(job, DeliveryLog.DeliveryStatus.FAILURE, e.getMessage());
            // Schedule retry or move to DLQ
            retryEngineService.handleFailure(job, e.getMessage());
            log.error("Job {} failed via {} - reason: {}",
                    job.getId(), job.getChannel(), e.getMessage());
        }
    }
 private void logAttempt(NotificationJob job,
```

```
                    DeliveryLog.DeliveryStatus status,
                    String response) {
        DeliveryLog log = new DeliveryLog();
        log.setJob(job);
        log.setAttemptNo(job.getRetryCount() + 1);
        log.setStatus(status);
        log.setResponse(response);
        log.setAttemptedAt(LocalDateTime.now());
        deliveryLogRepository.save(log);
    }
}
```




Send
Templates
History
Inbox
Analytics
Analytics
Delivery statistics for your account
Total Jobs
6
Delivered
6
Failed
0
Retrying
0
DLQ Size
0
Delivery Overview
Delivery Rate
100%
Failure Rate
0%
Pending Rate
0%
#14 EMAIL abcd@gmail.com COMPLETED 0 — 13 Apr 2026, 12:59 am
#15 EMAIL vivekg1234umbal@gmail.com COMPLETED 0 — 13 Apr 2026, 07:11 pm
#16 IN_APP vivek345 COMPLETED 0 — 13 Apr 2026, 07:24 pm
#17 IN_APP user1 COMPLETED 0 — 14 Apr 2026, 12:17 am
Showing 6 of 6 jobs
Analytics
Recipient
user@example.com
Subject
Notification subject
Message Body
Write your message here. Use {{placeholder}} for dynamic values.

NotifyAdmin
Notification Platform

Dashboard
Clients
Jobs
Dead Letter Queue
Analytics

Welcome, **Admin**

Logout

Dashboard

System overview at a glance

Total Clients	Total Jobs	Delivered	DLQ Size
8	17	13	0

Recent Jobs

Last 5 notification jobs

ID	CHANNEL	RECIPIENT	STATUS	RETRIES
#1	EMAIL	builproject55@gmail.com	FAILED	4
#2	EMAIL	buildproject55@gmail.com	COMPLETED	3
#3	EMAIL	buildproject55@gmail.com	COMPLETED	3
#4	EMAIL	builproject55@gmail.com	COMPLETED	3

NotifyAdmin
Notification Platform

Dashboard
Clients
Jobs
Dead Letter Queue
Analytics

Welcome, **Admin**

Logout

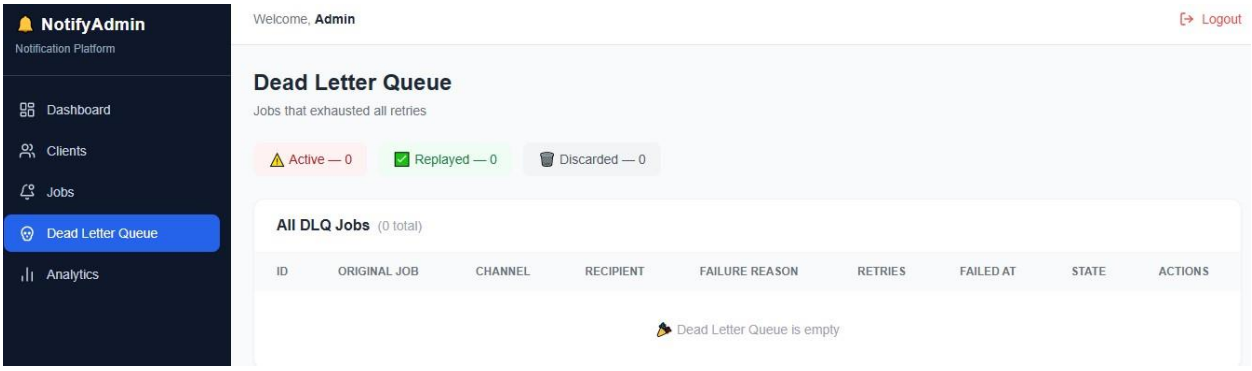
Clients

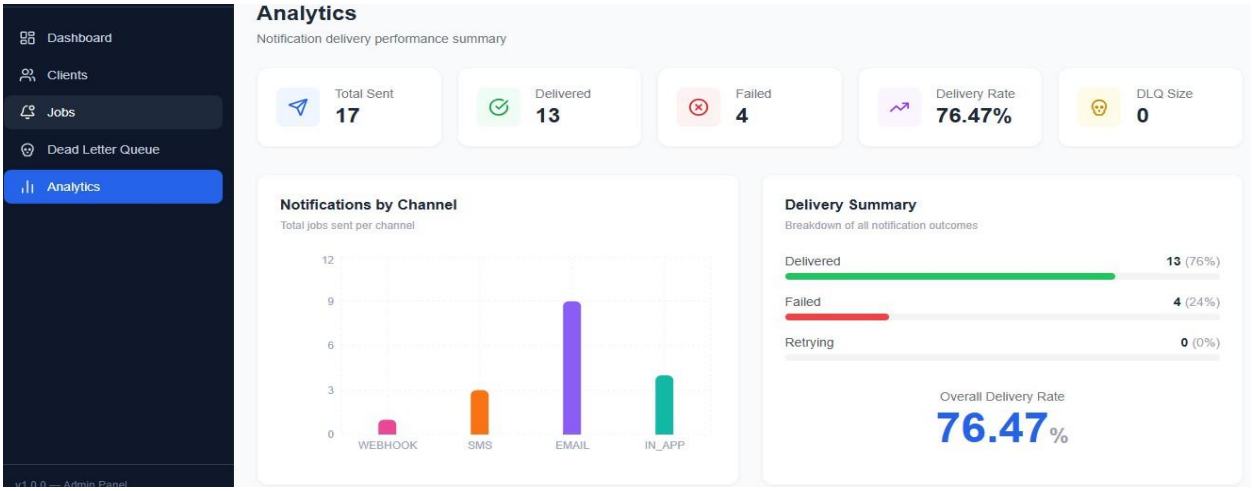
All registered clients — 8 total

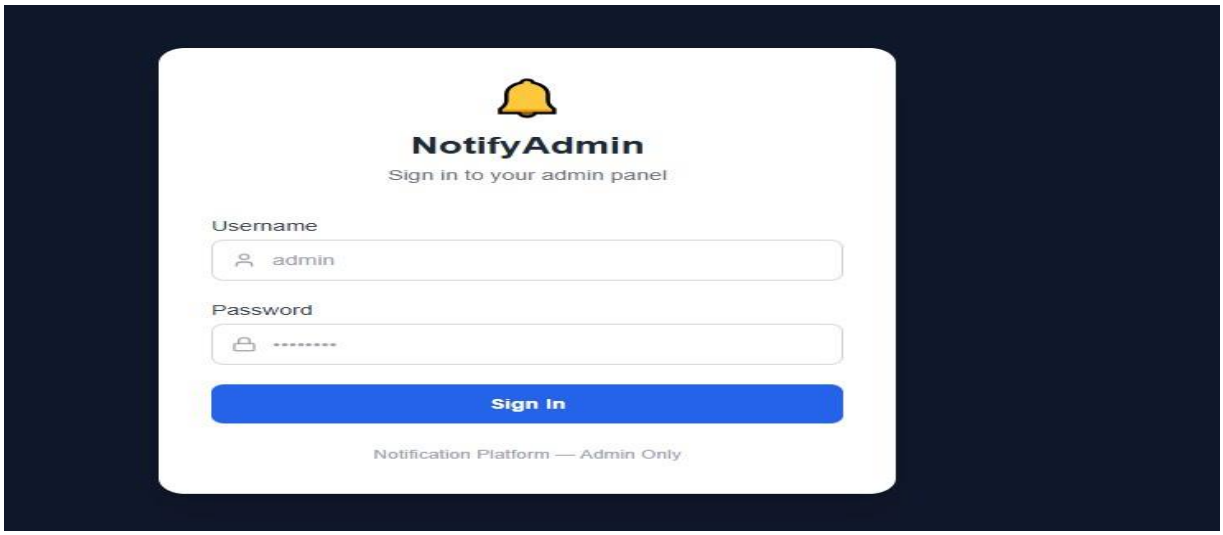
Register Client

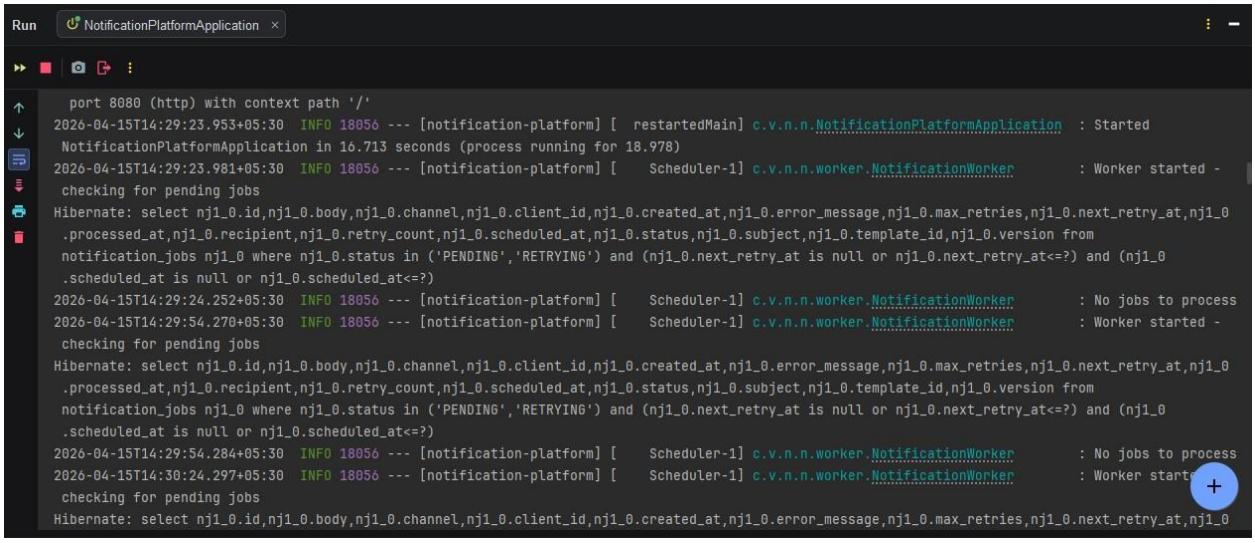
Registered Clients

ID	NAME	EMAIL	PLAN	STATUS	ACTION
#1	Test Company	testcompany@gmail.com	FREE	ACTIVE	Block
#2	shopEasy	shopeasy@gmail.com	FREE	ACTIVE	Block
#3	Zomato	zomato@gmail.com	FREE	ACTIVE	Block
#4	flipkart	flipkart@gmail.com	FREE	ACTIVE	Block
#5	swiggy	swiggy@gmail.com	FREE	ACTIVE	Block


NotifyAdmin
Notification Platform
Dashboard
Clients
Jobs
Dead Letter Queue
Analytics
Welcome, Admin
Logout
Dead Letter Queue
Jobs that exhausted all retries
Active — 0
Replayed — 0
Discarded — 0
All DLQ Jobs (0 total)
ID
ORIGINAL JOB
CHANNEL
RECIPIENT
FAILURE REASON
RETRIES
FAILED AT
STATE
ACTIONS
Dead Letter Queue is empty


Dashboard
Clients
Jobs
Dead Letter Queue
Analytics
Analytics
Notification delivery performance summary
Total Sent
17
Delivered
13
Failed
4
Delivery Rate
76.47%
DLQ Size
0
Notifications by Channel
Total jobs sent per channel
12
9
6
3
0
WEBHOOK
SMS
EMAIL
IN_APP
Delivery Summary
Breakdown of all notification outcomes
Delivered
13 (76%)
Failed
4 (24%)
Retrying
0 (0%)
Overall Delivery Rate
76.47%


NotifyAdmin
Sign in to your admin panel
Username
admin
Password
Sign In
Notification Platform — Admin Only


Run
NotificationPlatformApplication
port 8080 (http) with context path '/'
2026-04-15T14:29:23.953+05:30 INFO 18056 --- [notification-platform] [restartedMain] c.v.n.n.NotificationPlatformApplication : Started NotificationPlatformApplication in 16.713 seconds (process running for 18.978)
2026-04-15T14:29:23.981+05:30 INFO 18056 --- [notification-platform] [Scheduler-1] c.v.n.n.worker.NotificationWorker : Worker started - checking for pending jobs
Hibernate: select nj1_0.id,nj1_0.body,nj1_0.channel,nj1_0.client_id,nj1_0.created_at,nj1_0.error_message,nj1_0.max_retries,nj1_0.next_retry_at,nj1_0.processed_at,nj1_0.recipient,nj1_0.retry_count,nj1_0.scheduled_at,nj1_0.status,nj1_0.subject,nj1_0.template_id,nj1_0.version from notification_jobs nj1_0 where nj1_0.status in ('PENDING','RETRYING') and (nj1_0.next_retry_at is null or nj1_0.next_retry_at<=?) and (nj1_0.scheduled_at is null or nj1_0.scheduled_at<=?)
2026-04-15T14:29:24.252+05:30 INFO 18056 --- [notification-platform] [Scheduler-1] c.v.n.n.worker.NotificationWorker : No jobs to process
2026-04-15T14:29:54.270+05:30 INFO 18056 --- [notification-platform] [Scheduler-1] c.v.n.n.worker.NotificationWorker : Worker started - checking for pending jobs
Hibernate: select nj1_0.id,nj1_0.body,nj1_0.channel,nj1_0.client_id,nj1_0.created_at,nj1_0.error_message,nj1_0.max_retries,nj1_0.next_retry_at,nj1_0.processed_at,nj1_0.recipient,nj1_0.retry_count,nj1_0.scheduled_at,nj1_0.status,nj1_0.subject,nj1_0.template_id,nj1_0.version from notification_jobs nj1_0 where nj1_0.status in ('PENDING','RETRYING') and (nj1_0.next_retry_at is null or nj1_0.next_retry_at<=?) and (nj1_0.scheduled_at is null or nj1_0.scheduled_at<=?)
2026-04-15T14:29:54.284+05:30 INFO 18056 --- [notification-platform] [Scheduler-1] c.v.n.n.worker.NotificationWorker : No jobs to process
2026-04-15T14:30:24.297+05:30 INFO 18056 --- [notification-platform] [Scheduler-1] c.v.n.n.worker.NotificationWorker : Worker start
checking for pending jobs
Hibernate: select nj1_0.id,nj1_0.body,nj1_0.channel,nj1_0.client_id,nj1_0.created_at,nj1_0.error_message,nj1_0.max_retries,nj1_0.next_retry_at,nj1_0

CHAPTER 11

TESTING AND VALIDATION

11.1 Introduction

Testing is a crucial phase in the software development process that ensures the system works correctly and meets all requirements. It helps in identifying errors, improving system performance, and ensuring reliability.

In the **Multi-Channel Notification Delivery System**, testing is performed to verify that notifications are sent correctly through multiple channels, the retry mechanism works properly, and the system handles errors efficiently.

11.2 Objectives of Testing

The main objectives of testing are:

• To verify system functionality
• To identify and fix errors
• To ensure reliable notification delivery
• To validate system performance
• To ensure security and data integrity

11.3 Types of Testing

11.3.1 Unit Testing

Unit testing is performed on individual components or modules of the system.

Examples:

• Testing User Module
• Testing Notification Module
• Testing Channel Module

Purpose:

To ensure each module works independently without errors.

11.3.2 Integration Testing

Integration testing checks how different modules interact with each other.

Examples:

• Interaction between Notification Module and Channel Module
• Integration with Email and SMS APIs
Purpose:

To ensure smooth communication between modules.

11.3.3 System Testing

System testing evaluates the complete system as a whole.

Purpose:

• Verify overall functionality
• Check end-to-end workflow
• Ensure system meets requirements

11.3.4 User Acceptance Testing (UAT)

UAT is performed to check whether the system meets user expectations.

Purpose:

• Validate system usability
• Ensure user satisfaction

11.5 Validation Techniques

Validation ensures that the system meets user requirements and works as expected.

Input Validation

• Checks for valid message input
• Prevents empty or incorrect data

Output Validation

• Ensures correct delivery status
• Displays accurate results

Data Validation

• Ensures correct data storage
• Maintains data consistency

11.6 Performance Testing

Performance testing evaluates system efficiency under different conditions.

Parameters tested:
• Response time
• System load handling
• Processing speed

The system is capable of handling multiple notification requests efficiently.

11.7 Security Testing

Security testing ensures that the system is protected from threats.

Checks include:
• Unauthorized access prevention
• Secure API communication
• Data protection

11.10 Result Analysis

The testing results show that:

• Notifications are delivered successfully
• Retry mechanism works correctly
• System handles errors efficiently
• Data is stored accurately

11.12 Conclusion

Testing and validation confirm that the **Multi-Channel Notification Delivery System** works efficiently and meets all functional requirements. The system performs well under different conditions and ensures reliable notification delivery.

The successful testing of all modules ensures that the system is stable, secure, and ready for deployment.

CHAPTER 12

CONCLUSION

12.1 Summary of the Project

The **Multi-Channel Notification Delivery System** successfully demonstrates the development of a web-based communication platform that enables efficient and reliable notification delivery using modern technologies such as **Spring Boot, Hibernate, and MySQL**.

The system is designed to overcome the limitations of traditional notification systems by providing a centralized platform that supports multiple communication channels. It ensures that notifications are delivered through Email, SMS, and Push Notifications, improving the overall reliability and effectiveness of communication.

The system enables users and administrators to:
• Send notifications through multiple channels
• Track notification delivery status
• Manage user data efficiently
• Ensure reliable communication using retry mechanisms
• Maintain logs for monitoring and debugging

The project provides a complete solution for handling modern notification requirements in a structured and efficient manner.

12.2 Key Achievements

The project has successfully achieved the following milestones:

• Implementation of **multi-channel notification delivery system**
• Development of a **centralized notification management platform**
• Integration with **Email and SMS APIs**
• Implementation of **retry mechanism for failed notifications**
• Efficient **database design and management**
• Creation of a **user-friendly interface**
• Implementation of **logging and tracking system**

These achievements demonstrate the successful completion of all project objectives.

12.3 Learning Outcomes

Through the development of this project, several important technical and practical concepts were learned:

• Understanding of **Spring Boot framework** for backend development
• Knowledge of **Hibernate ORM** for database interaction
• Experience in **database design and normalization**
• Implementation of **multi-channel communication systems**
• Integration of **external APIs** for notification delivery
• Understanding of **system design and architecture**
• Experience in **testing, debugging, and error handling**

This project has provided hands-on experience in real-world application development.

12.4 Future Enhancements

The system can be further improved by adding advanced features:

• Implementation of **real-time notifications using WebSockets**
• Development of **mobile applications (Android/iOS)**
• Integration with **WhatsApp and Telegram notifications**
• Addition of **notification scheduling feature**
• Use of **AI-based intelligent notification routing**

• Implementation of **advanced analytics and reporting**
• Deployment on **cloud platforms for scalability**

These enhancements can transform the system into a large-scale enterprise solution.

12.5 Final Conclusion

The **Multi-Channel Notification Delivery System** successfully meets all its objectives by providing a reliable, efficient, and scalable solution for notification delivery. It demonstrates how modern technologies can be integrated to build a robust communication system.

The project effectively combines system design, database management, API integration, and error handling to create a complete working application. It ensures secure and efficient communication while maintaining simplicity and usability.

12.6 Practical Applications

The system can be applied in various real-world domains:

• **Banking systems** – for transaction alerts and OTP notifications
• **E-commerce platforms** – for order updates and delivery tracking
• **Educational institutions** – for announcements and notifications
• **Healthcare systems** – for appointment reminders and alerts
• **Business applications** – for system updates and internal communication

These applications highlight the usefulness of the system in real-world scenarios.

12.7 Project Impact

The project demonstrates how modern notification systems can be built using efficient and scalable technologies. It highlights the importance of reliable communication in digital applications.

The system provides:

• Practical understanding of real-world software development
• Improved knowledge of system design and architecture
• Hands-on experience with backend technologies
• Insight into building scalable and secure systems